DESIGN / ARCHITECTURE DOCUMENT

Explainable Adaptive Zero-Trust IDS for Multi-Tenant Smart-City Edge Simulation

Version 1.0 | Based on Project Technical Report, October 2026

Prepared from the supplied project technical report
Current state: Generation 1 MVP; future capabilities clearly marked

Document Control

Field	Description
Project Name	Explainable Adaptive Zero-Trust IDS for Multi-Tenant Smart-City Edge Simulation
Document Type	Design / Architecture Document
Version	1.0
Source Baseline	Comprehensive Project Status, Architectural Evolution & Technical Specification Report, October 2026
Current System State	Generation 1 (MVP) implemented as a Python simulation and evaluation pipeline
Intended Audience	Project team, academic reviewers, software developers, security engineers, evaluators

1. Introduction

1.1 Purpose

This document defines the logical architecture, components, data flows, trust evaluation, enforcement policies, audit mechanisms, interfaces, and verification approach for the Explainable Adaptive Zero-Trust IDS. It provides a design reference for understanding the current MVP and implementing the planned next generations.

1.2 Scope

The system simulates a shared municipal edge gateway serving Traffic Management, Water Utility, and Waste Management tenants. It evaluates device telemetry, detects anomalous behaviour using a hybrid machine-learning IDS, calculates a trust score, selects a policy action, applies simulated ingress enforcement, and records explainable security evidence.

The current scope is a software simulation. The report describes real packet filtering, cryptographic identity, real-world benchmark training, ONNX inference, FastAPI decoupling, and kernel-level eBPF/XDP enforcement as future work, not as completed MVP capabilities.

1.3 Design Goals

- Apply explicit, behaviour-based trust evaluation to every telemetry event.
- Detect suspicious deviations and attacks using a hybrid supervised/unsupervised ML pipeline.
- Translate trust scores into ALLOW, MONITOR, RESTRICT, or ISOLATE actions.
- Enforce tenant-aware containment while limiting disruption to unaffected tenants.
- Provide human-readable explanations and structured, persistent audit records.
- Measure detection quality, resource consumption, inference latency, and cross-tenant blast radius.
- Provide a staged path from a demonstrable MVP to an edge-native production architecture.

1.4 Terminology

Term	Meaning in this project
IDS	Intrusion Detection System; evaluates telemetry for attack-like or anomalous behaviour.
ZTA	Zero-Trust Architecture; devices are not trusted solely because of network location.
PEP	Policy Enforcement Point; applies the selected action to ingress traffic in the simulation.
PDP	Policy Decision Point; maps the calculated trust score to a policy action.
XAI	Explainable AI; provides feature evidence and readable decision explanations.

Tenant	A municipal workload domain: traffic, waste, or water.
Blast radius	Impact of containment on tenants other than the targeted tenant.
MVP / Gen 1	Current synthetic-data, Python-based implementation.
Gen 2 / Gen 3	Planned stateful and production-oriented evolutions, respectively.

2. System Context and Problem Statement

2.1 Operational Context

A shared Multi-Access Edge Computing (MEC) gateway hosts or forwards telemetry for three municipal tenants. The tenants have different traffic patterns and operational requirements: traffic devices are high-frequency and latency-sensitive; water utility devices carry mission-critical supervisory telemetry; waste sensors are typically low-frequency and resource-constrained.

2.2 Threat Scenario

The motivating scenario is a physically accessible smart waste-bin sensor, wst-002, becoming compromised and being used as a pivot toward other tenant networks. The simulated attack increases packet rate and payload size, contacts more destination IPs, targets the traffic subnet, and shifts from expected encrypted MQTT/TLS-like traffic to plaintext HTTP or RAW_TCP.

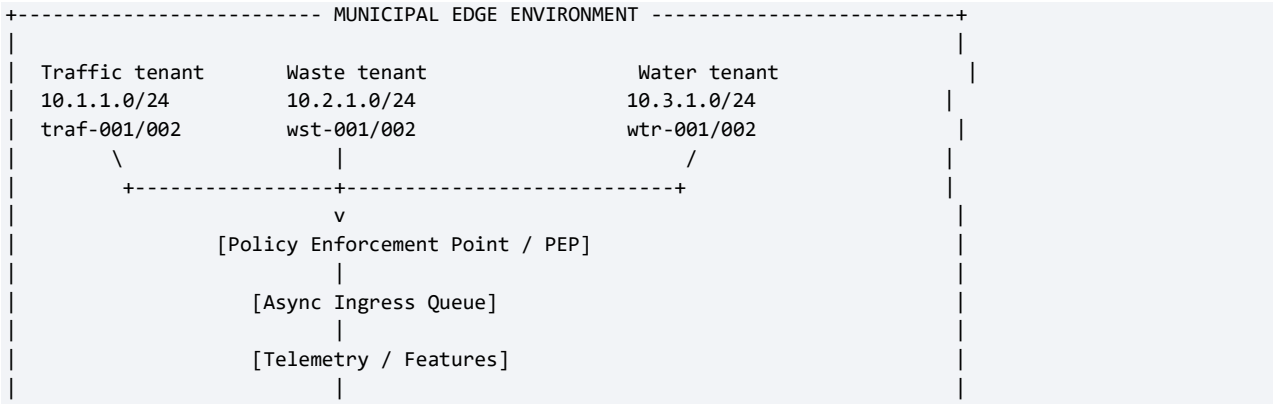
- Volumetric denial-of-service against shared gateway resources.
- Cross-tenant probing toward traffic or water devices.
- Malformed or unexpected payloads directed at shared telemetry services.
- Potential impersonation of devices where identity is only a plaintext device identifier (a current MVP limitation).

2.3 Zero-Trust Principles Applied

- Treat incoming device telemetry as untrusted regardless of subnet location.
- Continuously assess observed behaviour and derive a trust score in [0, 1].
- Separate policy decision logic (PDP) from enforcement logic (PEP).
- Apply tenant-aware containment and record the evidence behind each decision.
- Use graduated actions to avoid treating every anomaly as an immediate full isolation event.

3. Architecture Overview

3.1 Logical Architecture



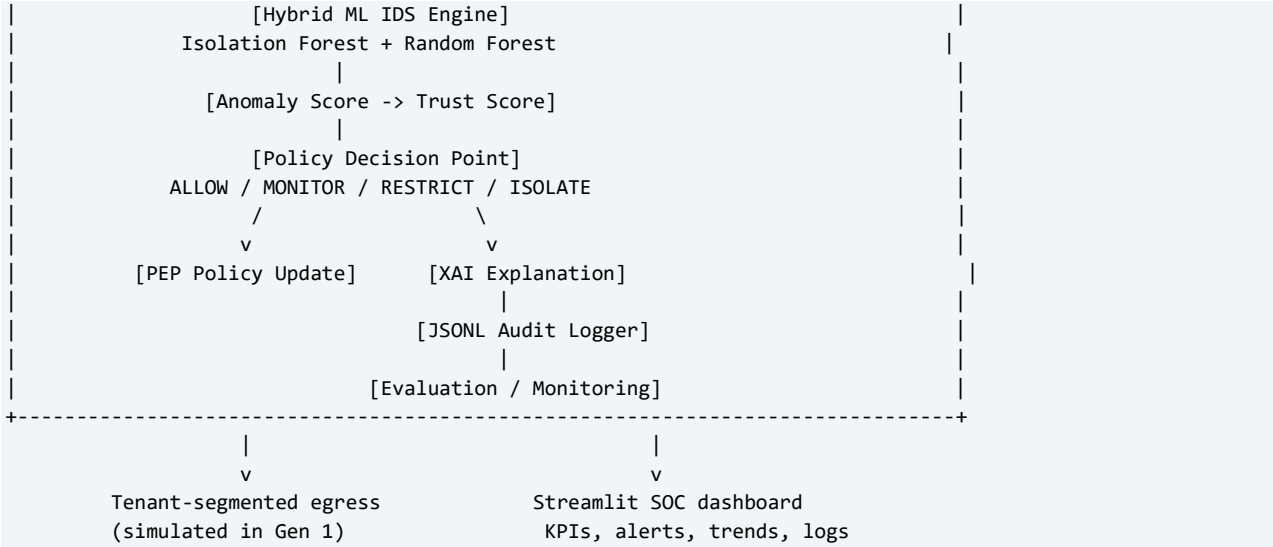


Figure 1. Logical architecture of the current Generation 1 prototype. The PEP, queue, IDS, PDP, XAI, and audit logger are implemented as Python components. The diagram is a logical view; it does not imply deployment of a production network firewall.

3.2 Architecture Layers

Layer	Responsibility	Current implementation
Simulation and input	Create normal device telemetry and inject attack behaviour.	src/simulation.py
Ingress / buffering	Provide an asynchronous, non-blocking event queue.	TrafficPipeline using asyncio.Queue
Schema and domain	Validate telemetry, decisions, actions, severity, and tenant types.	src/schemas.py; Pydantic v2
Detection and trust	Extract features; run Isolation Forest and Random Forest; calculate anomaly and trust scores.	src/ids_engine.py; scikit-learn
Policy decision	Map trust score to one of four policy actions.	PDP logic in src/ids_engine.py
Enforcement	Forward, monitor, rate-limit, or drop simulated ingress traffic.	src/enforcement.py
Explainability and audit	Generate readable reasons and persist structured audit records.	XAI logic and src/monitoring.py
Monitoring and evaluation	Track CPU, memory, inference latency, classification metrics, and containment impact.	src/monitoring.py; src/evaluation.py
Presentation and orchestration	Run full/demo modes and show security telemetry in a dashboard.	src/main.py; src/__main__.py; dashboard.py

3.3 Architectural Style

Generation 1 is a modular monolithic Python application: components have separate source files but run in one process and share the runtime. An asyncio queue models asynchronous ingestion. The Streamlit dashboard is part of the current project and the report describes it as an in-process/monolithic runtime. A standalone FastAPI service is planned for a later generation.

4. Actors, Tenants, and Device Model

4.1 System Actors

Actor	Role
IoT device / simulator	Produces telemetry attributed to a device and tenant.
ScenarioRunner	Introduces a compromised state and attack-like event patterns for wst-002.
IDS engine	Scores each event using the hybrid model.
PDP	Chooses the policy action from trust score thresholds.
PEP / ContainmentManager	Applies the policy action to simulated ingress.
Security operator	Uses dashboard, alerts, and audit records to inspect system behaviour.
Evaluation runner	Calculates detection metrics and cross-tenant containment results.

4.2 Tenant and Device Inventory

Tenant	Subnet	Device	Baseline packet rate	Baseline payload	Baseline destinations
Traffic	10.1.1.0/24	traf-001	12 packets/s	150 bytes	2
Traffic	10.1.1.0/24	traf-002	20 packets/s	512 bytes	3
Waste	10.2.1.0/24	wst-001	5 packets/s	64 bytes	1
Waste	10.2.1.0/24	wst-002	5 packets/s	64 bytes	1
Water	10.3.1.0/24	wtr-001	8 packets/s	96 bytes	2
Water	10.3.1.0/24	wtr-002	10 packets/s	128 bytes	2

The values above are the simulator's configured baseline profiles as reported in the source PDF; they are not measurements from a deployed municipal network.

4.3 Core Domain Types

- TenantType: traffic, waste, water.
- DeviceStatus: normal, compromised, isolated.
- PolicyAction: allow, monitor, restrict, isolate.
- AlertSeverity: info, warning, critical.

5. Data Model and Event Contracts

5.1 TelemetryEvent

Each incoming event carries an event identifier, timestamp, device identity, tenant, source and destination IP, traffic measurements, protocol/encryption indicators, and simulator ground truth where available.

```
TelemetryEvent (logical fields)
- event_id: unique event identifier
- timestamp: event time
- device_id: device identity string
- tenant: traffic | waste | water
- source_ip, dest_ip: source and destination addresses
- packet_rate: observed packet rate
- payload_size: observed payload size
- dest_count: number of destinations observed
- protocol: protocol label (e.g. MQTT, HTTP, RAW_TCP)
- is_encrypted: encryption indicator
- is_attack: simulator/evaluation ground truth
- attack_type: optional attack label
```

The report includes is_attack and attack_type in its synthetic event example. In a live deployment, ground-truth labels must not be treated as observable input features; they are for training/evaluation only.

5.2 TrustAssessment / PolicyDecision

```
PolicyDecision (logical fields)
- decision_id, timestamp
- device_id, tenant, event_id
- action: allow | monitor | restrict | isolate
- previous_action
- trust_score: [0, 1]
- anomaly_score: [0, 1]
- explanation: human-readable reason
- evidence:
    trust_score, anomaly_score
    feature_contributions
    action
```

5.3 AuditLogEntry

```
AuditLogEntry (logical fields)
- log_id, timestamp
- event_type, severity
- tenant, device_id, action
- explanation, evidence
- who: decision source (e.g. PDP-AutoPolicy)
- what: action or state change
- why: trigger or threshold explanation
```

5.4 Validation and Serialization

The report specifies Pydantic v2 schemas in src/schemas.py for strict typing and consistent contracts across components. The audit trail is stored as JSON Lines (JSONL) in data/audit_log.jsonl. The report does not provide a full formal API schema for all fields, so the field lists here are logical contracts based on the supplied examples.

6. End-to-End Data Flow

6.1 Normal Event Flow

1. The simulator emits a TelemetryEvent for one of the six devices.
2. The event is validated against the domain schema.
3. The simulated ingress path passes the event through the PEP and asynchronous queue.
4. The IDS extracts a seven-dimensional feature vector and applies the calibrated scaler.
5. The Isolation Forest and Random Forest branches produce anomaly-related scores.
6. The ensemble produces an anomaly score A; the system derives trust T = 1 − A.
7. The PDP maps T to ALLOW, MONITOR, RESTRICT, or ISOLATE.
8. The PEP applies the selected action to the simulated ingress traffic.
9. The XAI module creates a readable explanation with feature evidence and rule/threshold context.
10. The audit logger persists the decision; monitoring and evaluation collect metrics for dashboard/reporting.

6.2 Simulated Attack Flow

11. At 30% into the generated event stream, ScenarioRunner changes wst-002 to attack mode.
12. Packet rate increases to 3–8 times its baseline; payload size to 4–10 times baseline; destination count increases by 5–15; traffic may target the 10.1.1.x subnet.
13. Protocol behaviour shifts away from the expected encrypted MQTT/TLS-like pattern toward HTTP or RAW_TCP.
14. The detector evaluates the extracted features and computes anomaly and trust scores.

15. When trust falls below the relevant PDP threshold, the action escalates; a trust score below 0.35 maps to ISOLATE.
16. The PEP drops the target device's simulated ingress packets, and the decision is written to the audit log.
17. The evaluation checks whether the other tenant flows continue without simulated packet drops.

6.3 Data Flow Diagram

```
TelemetryEvent
     |
     v
Schema validation (Pydantic)
     |
     v
Feature extraction: [packet_rate, payload_size, dest_count,
                     is_encrypted, protocol_mqtt, protocol_http, protocol_raw]
     |
     v
StandardScaler (fit during edge calibration)
     |
     +-------------------------+
     |                         |
     v                         v
Isolation Forest          Random Forest
S_iso in [0,1]            P_RF(attack) in [0,1]
     |                         |
     +------------+------------+
                  v
        Ensemble anomaly score A
                  |
                  v
           Trust T = 1 - A
                  |
                  v
          PDP threshold mapping
                  |
                  v
    PEP action + XAI evidence + JSONL audit
```

7. Machine-Learning and Trust Architecture

7.1 Feature Vector

The current detector uses a seven-dimensional vector:

```
x = [packet_rate, payload_size, dest_count, I_encrypted, I_MQTT, I_HTTP, I_RAW_TCP]^T
```

The protocol and encryption indicators are represented numerically. The source report describes StandardScaler as being fitted during edge calibration and then applied before model inference.

7.2 Hybrid Detection Models

Branch	Model configuration	Purpose
Unsupervised	IsolationForest; 150 estimators; contamination=0.15	Detect statistical outliers without requiring an attack label for every observation.
Supervised	RandomForestClassifier; 200 estimators; max_depth=8	Estimate attack probability using labelled synthetic examples.
Scaling	StandardScaler	Normalize feature magnitudes using calibration data.

7.3 Ensemble and Trust Calculation

```
A = 0.60 * P_RF(attack | x) + 0.40 * S_IF(x)
T = 1.0 - A

A = anomaly score in [0, 1]
T = trust score in [0, 1]
P_RF = supervised Random Forest attack probability
S_IF = normalized Isolation Forest anomaly score
```

The source report specifies a blend weight of 0.60 for the Random Forest branch. The exact normalization of the Isolation Forest score should be verified against the implementation and unit tests before treating the formula as a calibrated probability.

7.4 PDP Thresholds

Trust score T	Policy action	Intended behaviour
T ≥ 0.80	ALLOW	Forward traffic normally.
0.60 ≤ T < 0.80	MONITOR	Forward traffic and flag it for observation/forensics.
0.35 ≤ T < 0.60	RESTRICT	Apply deterministic rate-limiting; current default passes 1 in N packets, N=3.
T < 0.35	ISOLATE	Drop all simulated ingress packets for the isolated device.

7.5 Explainability

Generation 1 attaches normalized feature importance/evidence and generates deterministic human-readable explanations. The report's example identifies packet rate, payload size, destination count, encryption, and protocol indicators as possible contributors. The exact local feature-contribution method should be described as implementation-specific unless the code confirms it.

Generation 2 plans hybrid TreeSHAP attribution and tenant-specific rolling baseline Z-scores. Generation 3 proposes counterfactual explanations and automated MITRE ATT&CK mapping.

8. Policy Enforcement and Tenant Isolation

8.1 Current PEP Behaviour

Action	Current simulated behaviour
ALLOW	Traffic forwards unmodified.
MONITOR	Traffic forwards with forensic telemetry flagging.
RESTRICT	Deterministic rate limiting: passes 1 in N packets; default N=3.
ISOLATE	Immediate simulated ingress packet drop.

8.2 Blast-Radius Metric

```
BlastRadius(tenant_i) = DroppedPackets(tenant_i) / TotalPackets(tenant_i)
```

The required invariant is that isolating a waste device does not drop packets for the unaffected Traffic and Water tenants. The report states that this property is programmatically validated in the simulation. It should be revalidated under more diverse attack loads and, for production claims, with real traffic enforcement tests.

8.3 Planned Enforcement Evolution

- Gen 2: token-bucket rate limiting, command stripping/read-only status mode, signed device tokens, and token revocation on isolation.
- Gen 3: O(1) in-memory policy lookup on the forwarding fast path and kernel-level eBPF/XDP packet enforcement.
- These are planned capabilities and must not be described as implemented in the current MVP.

8.4 Fail-Safe Considerations

The source report does not specify production fail-open/fail-closed behaviour, recovery procedures, administrative override, or availability guarantees. These decisions must be made before deployment on real traffic, particularly because Traffic and Water services may be safety-critical.

9. Explainability, Auditing, and Observability

9.1 Audit Pipeline

Each significant policy decision should produce a structured audit record containing who/what/why context, tenant and device identity, timestamp, selected action, trust/anomaly values, explanation, severity, and evidence. The current persistence target is data/audit_log.jsonl.

9.2 Resource and Performance Monitoring

- CPU utilization through psutil.
- Resident memory footprint in MB.
- Rolling inference latency in milliseconds.
- Detection metrics: precision, recall, F1-score, and false-positive rate.
- Policy action distributions, device/fleet state, trust trends, alerts, and recent audit records in the Streamlit dashboard.

9.3 Incident Explanation Contract

A useful incident explanation should identify the device and tenant, action and trust/anomaly scores, strongest feature deviations, any cross-tenant violation, the policy rule/threshold that triggered, and the action applied. The detailed TreeSHAP/Z-score incident card in the source report is a future-state design example, not proof that TreeSHAP is active in Generation 1.

10. Software Components and Repository Layout

Path / module	Responsibility
src/__init__.py	Package namespace.
src/__main__.py	Module entry point.
src/schemas.py	Pydantic domain schemas and enums.
src/simulation.py	Multi-tenant device simulator, attack scenario runner, and asynchronous pipeline.
src/ids_engine.py	Hybrid IDS, trust score, PDP logic, and explainability functions.
src/enforcement.py	PEP action handling and containment manager.
src/monitoring.py	JSONL audit logging and psutil performance monitoring.
src/evaluation.py	Benchmark metrics and Markdown report generation.
src/main.py	CLI orchestration for full and demo modes.
tests/test_ids.py	Unit and integration tests; report states 25 tests pass.
reports/evaluation_report.md	Generated benchmark report.
data/audit_log.jsonl	Persistent structured audit trail.
dashboard.py	Streamlit security monitoring dashboard.

requirements.txt	Dependency/version specification.
ARCHITECTURE.md	Mermaid architecture specification.
PLAN.md	Six-phase implementation tracking plan.
README.md	Setup and CLI quick-start instructions.

10.1 Runtime Entry Points

The report identifies a unified CLI with full and demo modes, launched through the Python package entry point. The Streamlit dashboard is a separate presentation entry point. Exact command-line flags, endpoint routes, and dashboard controls are not specified in the source report and should be taken from README.md or the implementation rather than assumed.

11. Technology Stack

Technology	Use in project	Status in report
Python 3.13.14	Runtime for simulation, ML, orchestration, and CLI.	Active
asyncio	Non-blocking event queue and async tasks.	Current
Pydantic 2.14.1	Typed schema validation.	Current
scikit-learn 1.9.1	Isolation Forest, Random Forest, StandardScaler.	Current
NumPy 2.2.3	Numerical feature processing.	Current
Pandas 3.0.6	Tabular data and evaluation.	Current/planned use
Streamlit 1.65.0	SOC-style monitoring dashboard.	Current
Rich 15.0.0	Formatted CLI output.	Current
psutil 7.2.2	CPU, memory, and process monitoring.	Current
pytest 9.1.1	Automated unit/integration testing.	Current
joblib 1.6.0	Model serialization.	Listed in stack
FastAPI 0.143.0 + Uvicorn 0.54.0	Standalone API/WebSocket service.	Planned Gen 2/3; report labels API as Phase 7
SHAP	Tree-based feature attribution.	Planned Gen 2
PyJWT / cryptography	Signed device tokens and identity claims.	Planned Gen 2
ONNX Runtime	Compiled inference and micro-batching.	Planned Gen 3
eBPF/XDP, Docker, K3s	Kernel enforcement, constrained deployment, orchestration.	Planned Gen 3

12. Non-Functional Requirements and Quality Attributes

Quality attribute	Design requirement / source baseline
Security	Zero-trust decisioning, tenant-aware policy actions, explainable evidence, audit trail. Production device authentication is a future upgrade.
Isolation	Contain a compromised waste device without packet drops for unaffected tenants in the simulation.
Performance	Measure inference latency, CPU, and memory. Gen 1 is reported at roughly 19–39 ms per single sample in the limitations discussion.
Scalability	Async queue in Gen 1; planned batching and O(1) fast-path policy lookup in later generations.
Observability	Trust trends, policy actions, alerts, audit records, and resource metrics.
Maintainability	Separation into schemas, simulation, IDS, enforcement, monitoring, evaluation, and UI modules.

Explainability	Every policy decision should be associated with feature evidence and a readable reason.
Testability	Automated schema, simulation, ML, PDP, PEP, and cross-tenant validation tests.
Portability	Python-based prototype; constrained Linux/ARM-style deployment is a future target, not established by the current report.

The report gives design targets and observed prototype claims, but does not define formal service-level objectives for availability, throughput, queue backpressure, recovery time, or maximum false-positive rate. These should be specified and measured before production use.

13. Testing and Acceptance Criteria

13.1 Existing Verification Reported

- 25 pytest unit and integration tests are reported as passing.
- Coverage areas include schemas, simulation, ML models, PDP, PEP, and cross-tenant validation.
- The report claims precision=1.0, recall=1.0, and F1=1.0 on the synthetic scenarios.
- The report claims zero blast radius for unaffected Traffic and Water tenants in the simulation.

13.2 Recommended Acceptance Tests

18. Schema tests: reject invalid tenant/action values and malformed telemetry fields.
19. Baseline simulation: all six devices produce telemetry within configured normal profiles.
20. Attack injection: verify wst-002 changes behaviour at the configured 30% stream point.
21. Detection: calculate precision, recall, F1, and false-positive rate on a held-out split; ensure no train/test leakage.
22. Policy boundaries: test values immediately below, at, and above 0.35, 0.60, and 0.80.
23. PEP action tests: ALLOW forwards, MONITOR flags, RESTRICT applies the configured sampling/rate-limit, ISOLATE drops target ingress.
24. Tenant isolation: isolating waste traffic must not cause drops for traffic or water traffic.
25. Audit integrity: each policy change produces a parseable JSONL entry with decision, reason, and evidence.
26. Performance: record latency distributions (median, p95, p99), CPU, memory, and queue depth under increasing load.
27. Robustness: test bursty traffic, low-and-slow attacks, benign traffic shifts, missing fields, malformed events, and recovery after false alarms.

13.3 Interpretation of Synthetic Results

Perfect classification on the current synthetic data demonstrates that the pipeline works for the generated scenarios; it does not establish equivalent real-world accuracy. The source report itself identifies simplified distributions and deterministic attack signatures as limitations. Realistic datasets, held-out evaluation, and adversarial scenarios are required before claiming production detection effectiveness.

14. Security and Operational Considerations

- Keep simulator ground-truth labels out of live inference features and operational decision logic.
- Protect audit files against unauthorized modification and define retention/access policies before deployment.
- Validate device-to-tenant mapping at ingress; do not trust a plaintext device_id alone in production.
- Define secure model artifact loading, versioning, and rollback procedures.
- Set explicit queue limits and backpressure behaviour so a flood cannot exhaust memory.

- Specify policy transition hysteresis and a safe recovery/re-key process before enabling automatic isolation on real systems.
- Use least privilege for the runtime and dashboard; separate read-only monitoring from privileged enforcement controls.
- Test fail-open/fail-closed behaviour and administrative override for safety-critical municipal services.
- Treat all stated network addresses, attack labels, and performance figures as simulation/report values unless independently validated.

These items are architecture recommendations derived from the system's threat model and the gaps identified in the source report. They are not represented as already implemented.

15. Deployment Architecture and Evolution Roadmap

15.1 Generation 1 — Current MVP

- Synthetic multi-tenant telemetry and scenario-driven attack injection.
- Stateless hybrid scikit-learn IDS and static PDP thresholds.
- Python-level simulated PEP and containment manager.
- Deterministic XAI explanations, JSONL auditing, Streamlit dashboard, and Markdown evaluation report.
- Unconstrained host-process deployment in the report; no production network enforcement implied.

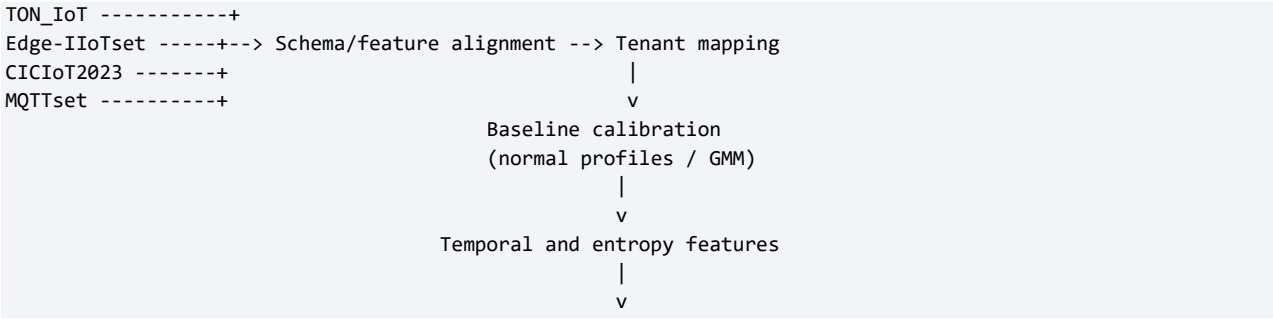
15.2 Generation 2 — Stateful and More Explainable Zero Trust

- Train/calibrate against TON_IoT, Edge-IIoTset, CICIoT2023, and MQTTset data, with tenant-aware mappings.
- Introduce asymmetric EWMA trust memory: rapid penalty for suspicious behaviour and slower recovery.
- Use hysteresis to reduce policy flapping.
- Add TreeSHAP and rolling tenant baseline Z-scores.
- Add signed HMAC-SHA256 or Ed25519 device tokens with tenant claims.
- Upgrade restriction to token-bucket rate limiting and command filtering.
- Decouple the IDS engine as a FastAPI service as planned in the roadmap.

15.3 Generation 3 — Edge-Native Production Objective

- Use a constant-time policy-state lookup on the forwarding fast path.
- Micro-batch telemetry (target batch size 64 or 50–100 ms window, per report) and run ONNX-compiled inference.
- Use an autonomous API service and WebSocket metric/alert stream.
- Validate under Docker CPU/memory constraints; the report gives 0.5 CPU and 256 MB as a target example.
- Explore eBPF/XDP enforcement and K3s orchestration.
- Target the report's stated inference objective of less than 1.5 ms per 64-event batch, subject to benchmarking on specified hardware.

15.4 Proposed Dataset Pipeline



```
Calibrated model training
        |
        v
Replay through async pipeline
        |
        v
Metrics + tenant blast-radius tests
```

The dataset choices and mapping are proposals in the report. Dataset licensing, preprocessing details, feature availability, label harmonization, and reproducible train/validation/test partitions must be established during implementation.

16. Design Decisions, Assumptions, and Open Questions

16.1 Design Decisions Documented in the Source

- Three tenant domains share an edge-gateway simulation.
- Seven numeric telemetry features feed a hybrid Isolation Forest / Random Forest detector.
- Random Forest attack probability has ensemble weight 0.60.
- Trust is defined as one minus the anomaly score.
- Four policy actions are selected by fixed trust thresholds.
- PEP enforcement and cross-tenant validation are separate from model scoring.
- JSONL audit records and a Streamlit dashboard support observability.

16.2 Assumptions

- Configured device baselines approximate expected normal behaviour in the simulator.
- Protocol/encryption indicators are available to the feature extractor in the simulated event stream.
- The synthetic scenario labels are available for supervised training and offline evaluation.
- The current implementation's claim of successful isolation is limited to the simulator and its tests.

16.3 Open Questions Before Production

- What is the exact normalization of the Isolation Forest output, and how is it calibrated with Random Forest probabilities?
- Are feature importance values global model importances or local per-event explanations?
- How will queue overload, event loss, model exceptions, and audit-write failures be handled?
- What are the policy transition, recovery, manual override, and emergency allow-list rules?
- How will cryptographic keys, token expiry, revocation, replay protection, and device re-keying be managed?
- What are the measured throughput, p95/p99 latency, false-positive rate, and CPU/RAM limits on target hardware?
- Which API endpoints and WebSocket event contracts will the planned FastAPI service expose?
- How will real benchmark datasets be split to prevent device/attack leakage and enable reproducible evaluation?
- What evidence is required before asserting zero performance impact on real tenant networks?

17. Conclusion

The design is a modular Python-based, explainable zero-trust IDS simulation for a shared smart-city edge gateway. It combines synthetic multi-tenant telemetry, a hybrid Isolation Forest/Random Forest detector, trust-score-based policy decisions, simulated enforcement, tenant-isolation checks, structured audit logging, and a Streamlit monitoring dashboard. The report describes the Generation 1 MVP as implemented and tested on synthetic scenarios.

The most important architectural distinction is between the current verified simulation and the proposed production direction. Stateful trust, real benchmark training, cryptographic identity, TreeSHAP, FastAPI decoupling, ONNX batching, constrained containers, and eBPF/XDP are roadmap items. They should be implemented and independently evaluated before the system is described as production-ready for live municipal infrastructure.

Appendix A — Core Decision Reference

Input condition	Decision / result
Trust T ≥ 0.80	ALLOW
0.60 ≤ T < 0.80	MONITOR
0.35 ≤ T < 0.60	RESTRICT
T < 0.35	ISOLATE
Attack scenario begins	At 30% of the event stream for wst-002
Restriction default	Pass 1 in every 3 packets (N=3)
Model blend	A = 0.60 × Random Forest attack probability + 0.40 × normalized Isolation Forest score
Trust mapping	T = 1 − A
Feature vector dimension	7
Isolation Forest	150 estimators; contamination 0.15
Random Forest	200 estimators; max_depth 8
Reported automated tests	25 passing
Reported synthetic F1	1.0000 (synthetic scenarios only)

Appendix B — Source Boundary

This document is a structured architecture specification derived from the uploaded 15-page project report. It is not a source-code audit: the PDF describes repository modules and claims test outcomes, but the actual Python source files were not provided for line-by-line verification. Where the PDF is silent, this document labels the detail as an open question or recommendation instead of asserting implementation.